CS1312
Algorithms Design and Analysis

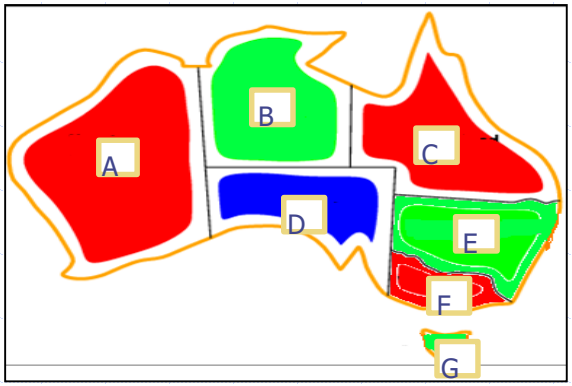
P, NP, and NP-Complete, NP-Hard Problems & Approximation Algorithms

Introduction

- Problems we are trying to solve are basically of two kinds:
  - **Decision problems**: trying to decide whether a statement is true or false.
    - They have either **yes** or **no** as the solution.
  - **Optimization problems:** trying to find the solution with the best possible score according to some scoring scheme.
    - **Maximization problems:** trying to maximize a certain score.
    - **Minimization problems:** trying to minimize a cost function.

Introduction

- Example: graph coloring problem:
  - Given a graph, how to color vertices so that adjacent ones have different colors.



  - **Chromatic number is the smallest number of colors needed to color a graph**.
  - **Optimization problem:** Given a graph G=(V,E), find the chromatic number.
  - **Decision problem:** Given a graph G=(V,E) and an integer k, is G k-colorable?

Introduction

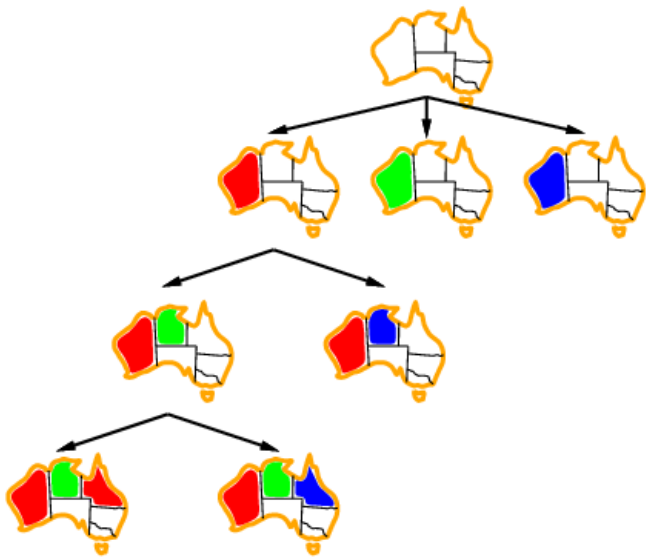
- We will focus on the *decision problems* because solving general problem is as hard as solving the decision problem version.

Tractable and intractable problems

- Some problems are intractable
  - They grow large, we are unable to solve them in reasonable time.
- Polynomial-Time Algorithm
  - Tractable
  - On inputs of size n, their worst-case running time is $O(n^k)$ for some constant k.
  - O(1), O(log n), O(n), O(n log n), $O(n^2)$, $O(n^3)$
- Exponential-Time Algorithm
  - Intractable
  - On inputs of size n, algorithms which are not bounded from above by a polynomial function of n.
  - $O(2^n)$, O(n!), $O(n^n)$

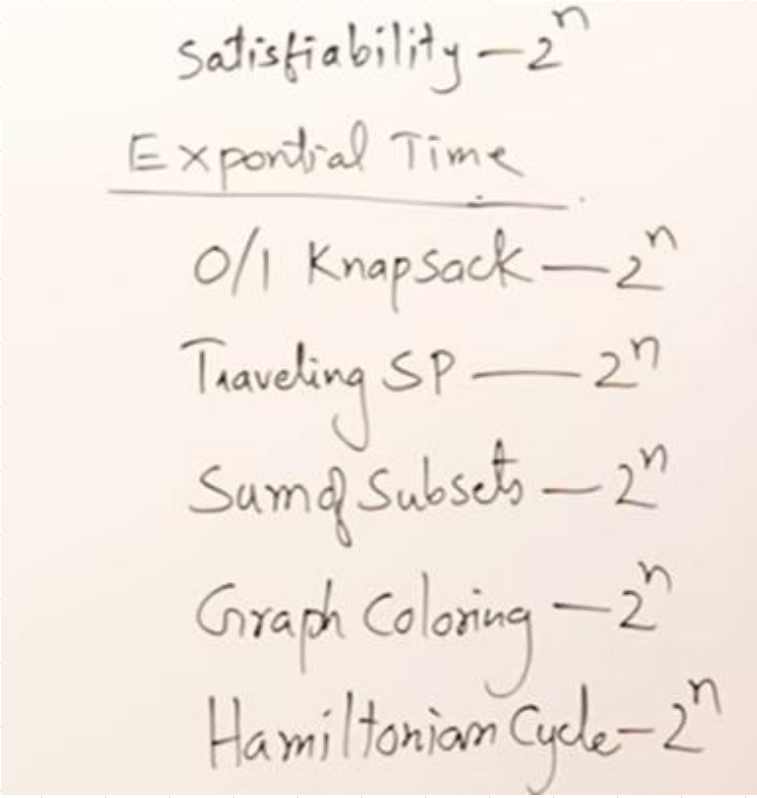
Tractable and **intractable** problems

- Example of *exponential time* is the **graph coloring problem:**
  - Time complexity: $O(d^n)$.
  - Example: n=50 and d=2, time complexity : 2^{50}
  - If each case takes 1 ms, 2^{50} ms = **35678 years**



Tractable and intractable problems

- Example of *exponential time* is 3-CNF-SAT problem.
- What is the satisfiability (SAT) problem?

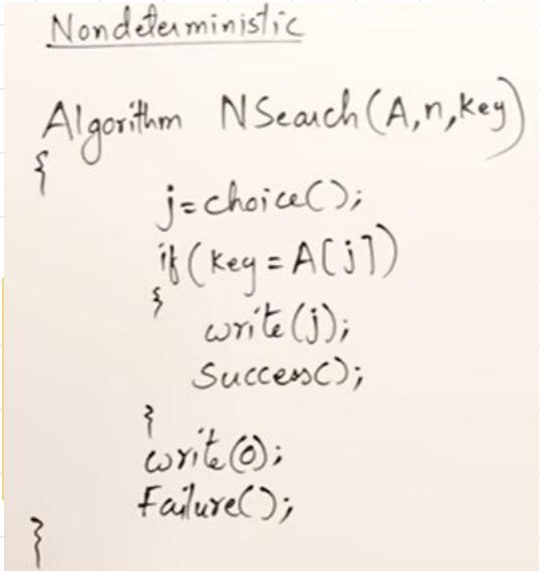


P versus NP problem

- The class **P** consists of all those problems that are **solvable** in polynomial time.
  - The problems that can be solved in time $O(n^k)$ for some constant k, where n is the size of the input to the problem.
  - Examples:
    - Sorting; merge sort, heapsort, insertion sort, etc.
    - Searching; linear or binary
    - Matrix multiplications
    - Single-source shortest path problem; Bellman-ford and Dijkstra algorithms

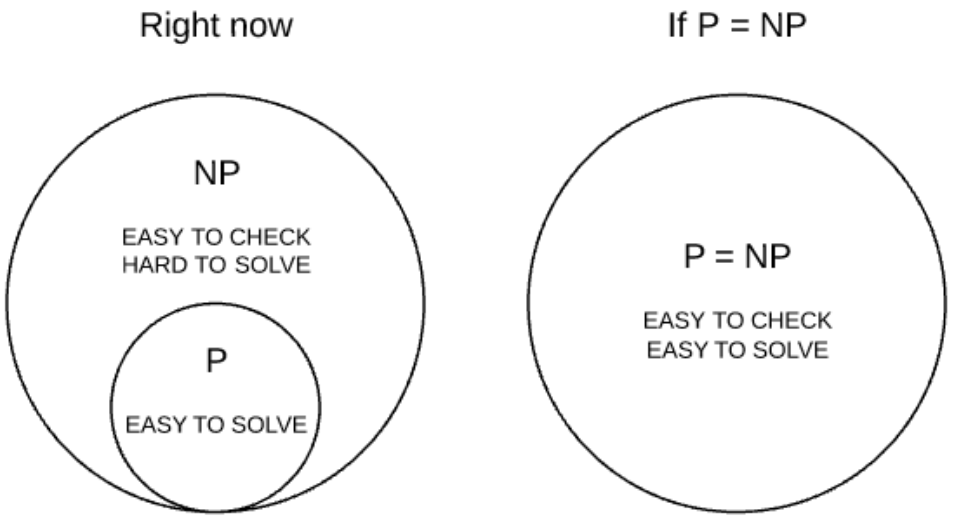
P versus NP problem

- The class **NP** consists of all those problems that are **verifiable** in polynomial time.
  - Examples:
    - 3-CNF-SAT problem, Hamiltonian cycle problem, etc.
  - NP is stands for nondeterministic polynomial.
  - NP is the class of decision problems that are solvable in polynomial time with a nondeterministic algorithm.
  - Non-deterministic algorithm
    - Example-Search problem:
    - Check if X found in a set (1:n)?

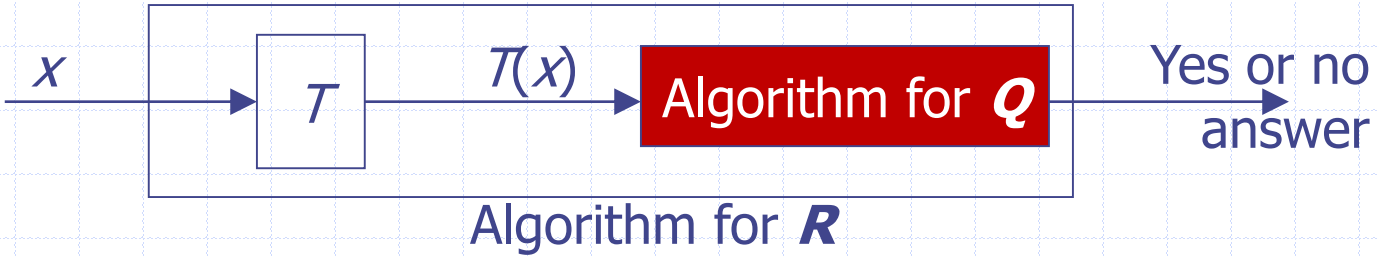


P versus NP problem

- $P \subseteq NP$
- Is P = NP ? Is p is a proper set of NP?
  - Whether every problem whose solution can be quickly verified can also be quickly solved?
  - Open question since 1970
  - Millennium Prize Problems

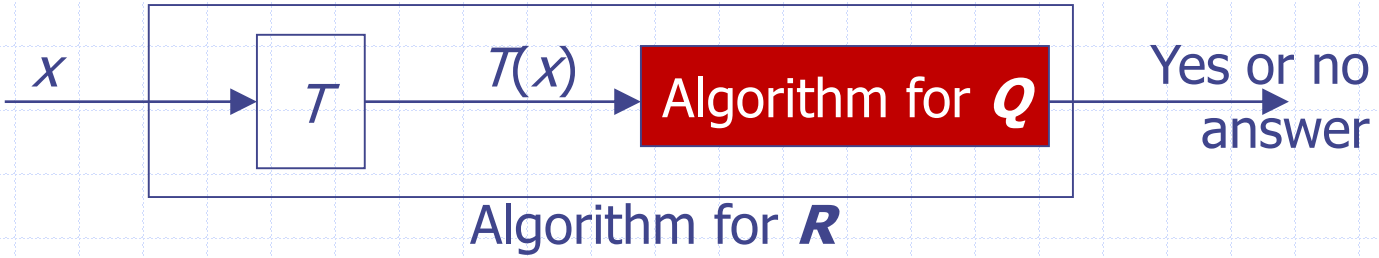


Polynomial-time Reductions



- Problem R is polynomially reducible to Q if such a transformation T can be computed in *polynomial time*.
- Correct answer for R on x is "yes", iff the correct answer for Q on T(x) is "yes".
- $R \leq_p Q$ means that R is *polynomial-time reducible* to Q
- R is not more than a polynomial factor harder than Q.
- If $L1 \leq_p L2$ and $L2 \leq_p L3$, then $L1 \leq_p L3$ (*Transitivity*)

Polynomial-time Reductions

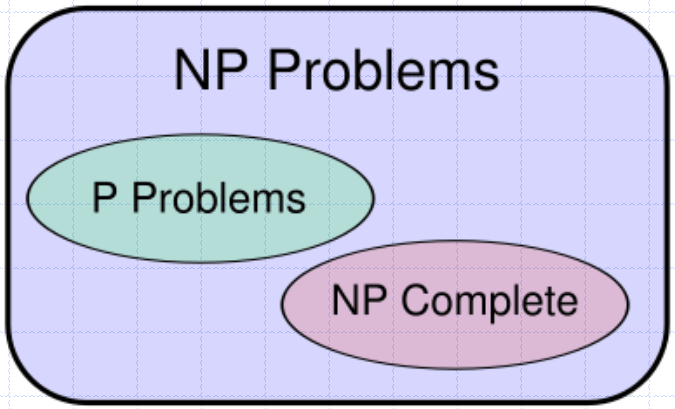


- **Theorem**:
  - If R $\leq_p$ Q and Q is ***P***, R is also ***P***.
- **Theorem**:
  - If R $\leq_p$ Q and R is ***NP***, Q is also ***NP***.

NP-complete problem

- A problem is in the class **NP-complete (NPC)** if it is in NP and is as "hard" as any problem in NP.
  - The hardest problems in NP.
  - 3-SAT problem, Hamiltonian cycle problem, longest path problem, etc.

- A decision problem *Q* is *NP*-complete if:
  1. $Q \in NP$
  2. for all $R \in NP$, $R \leq_p Q$

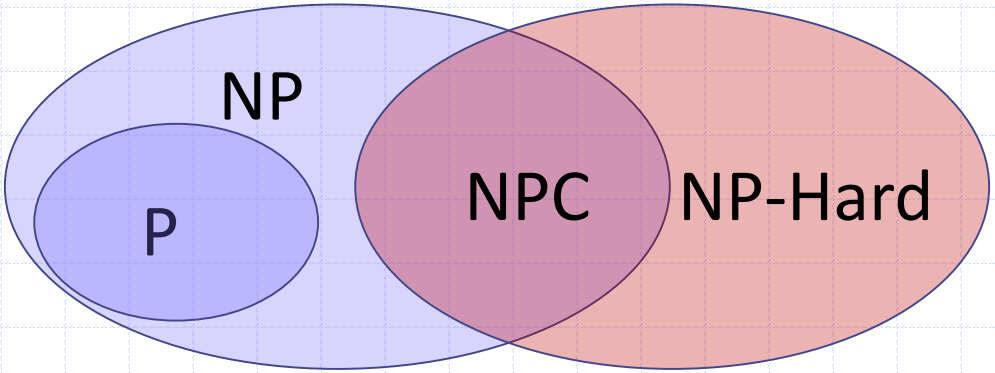


Solving NP-complete problems

- We use reductions (or transformations) to prove that a problem is NP-complete.
- How to Prove a Problem **L** is **NP-Complete**?
  - Prove **L** $\in$ **NP.**
  - Select a known NP-complete problem **L'**.
  - Describe an algorithm that computes a function ***f*** mapping every instance $x \in \{0,1\}^*$ of **L'** to an instance ***f(x)*** of **L**.
  - Prove that the function ***f*** satisfies $x \in$ **L'** if and only if ***f(x)*** $\in$ **L** for all $x \in \{0,1\}^*$.
  - Prove that the algorithm computing ***f*** runs in polynomial time.

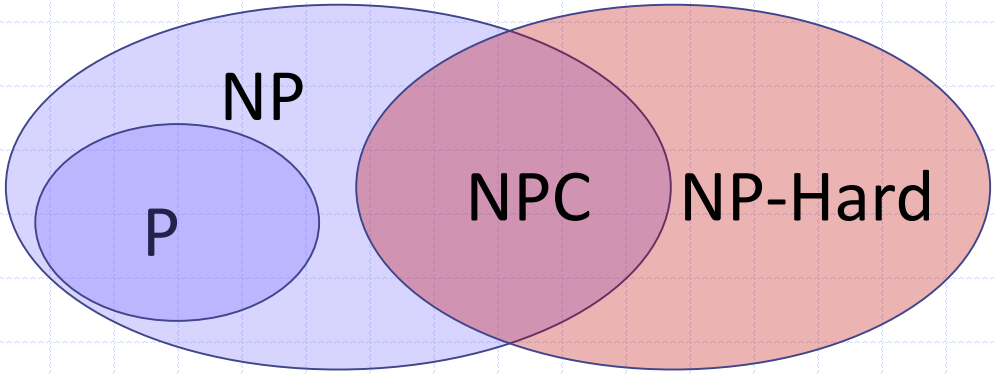
NP-Hard problems

- At least as hard as the hardest problems in NP.
- If **all** problems in NP are polynomial-time reducible to Q, then Q is **NP-Hard**.
- It is **not** necessarily that **NP-Hard** $\in$ NP.



- **Theorems:**
  - If $R \leq_p Q$ and R is ***NP-Hard***, Q is also ***NP-Hard***.
    - $\leq$ is transitive.

NP-Hard and NP-complete problems



- **Theorems**:
  - If $R \leq_p Q$ and R is ***NP-Complete***, then Q is ***NP-Hard***.
  - If $R \leq_p Q$ and R is ***NP-Complete*** and Q is ***NP***, then Q is ***NP-Complete.***

NP-complete problem

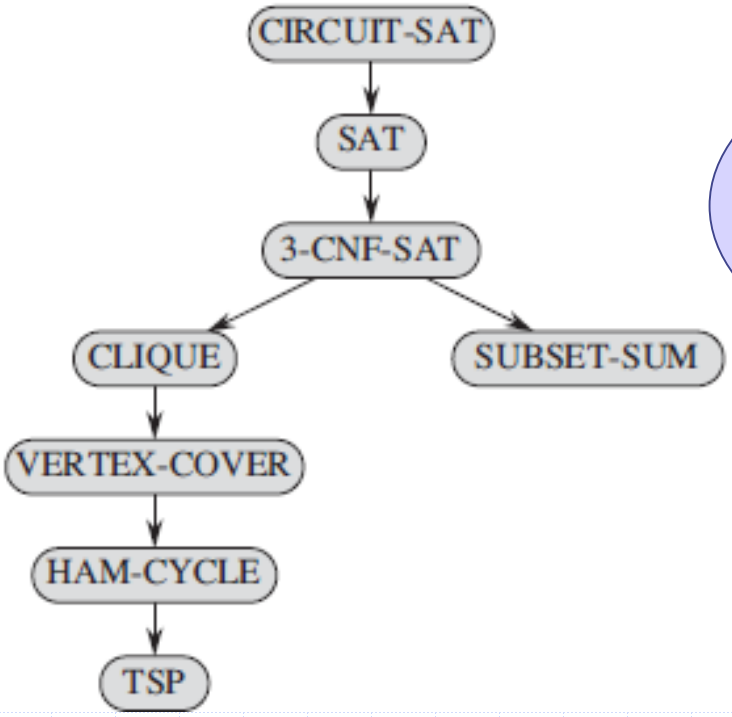
- Interesting property
  - If any one **NP-complete** problem can be solved in polynomial time, then every problem in NP can also be solved similarly.

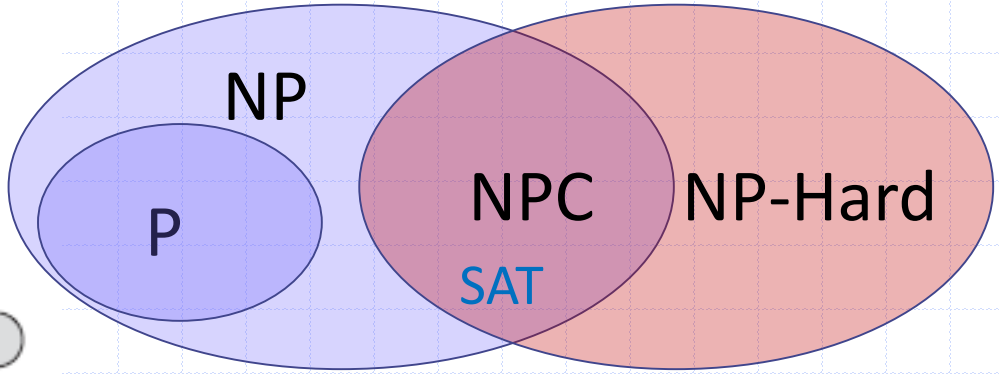
NP-Completeness Proofs

- We can prove a new problem **Q** is NP-complete by reducing known an **NP-complete** problem **R** to it.
  - This will implicitly reduce every problem in **NP** to **Q**.
  - Transitivity
- Example:
  - 3-CNF-SAT$\leq_p$ CLIQUE

NP-Completeness Proofs

- We will use the fact that CIRCUIT-SAT is ***NP-complete*** to prove a few other problems that are NP-Complete. This is based on "Reducibility".





NP-Completeness Proofs

- We need to find some first NP-complete problem.
  - Finding the first NP-complete problem was the result of the Cook-Levin theorem.
  - Boolean satisfiability (SAT) problem is NP-complete.
  - The theorem is named after Stephen Cook and Leonid Levin.

The Satisfiability (SAT) Problem

- Given a Boolean expression on *n* variables, can we assign values such that the expression is TRUE?
- Example:
  - $(x_1 \wedge \neg x_1)$ is unsatisfiable.
  - $(x_1 \wedge x_2)$ is satisfiable.
  - $((x_1 \wedge x_2) \vee \neg((\neg x_1 \wedge x_3) \vee x_4)) \wedge \neg x_2$ is satisfiable.

X_1	X_2	X_3	X_4	Expression
?	?	?	?	T

- Seems simple enough, but no known deterministic polynomial time algorithm exists.
- Easy to verify in polynomial time!

Conjunctive Normal Form (CNF)

- boolean variables: take on values T or F
  - Ex: x, y
- literal: variable or negation of a variable
  - Ex: x, $\overline{x}$
- clause: disjunction (OR) of several literals
  - Ex: $x \vee y \vee \overline{z} \vee w$
- CNF formula: conjunction (AND) of several clauses
  - Ex: $(x \vee \overline{y}) \wedge (\overline{z} \vee w \vee x)$

Satisfiable CNF Formula

- Is $(x \vee y)$ satisfiable?
  - yes: set x = T and y = F to get overall T
- Is $\overline{x} \wedge x$ satisfiable?
  - no: both x = T and x = F result in overall F
- Is $(\overline{x} \vee y) \wedge (\overline{z} \vee w \vee x)$ satisfiable?
  - yes: x = T, y = T, z = F, w = T result in overall T
- If formula has n variables, then there are 2^n different truth assignments.

The Satisfiability (SAT) Problem

- SAT = all (and only) strings that encode satisfiable CNF formulas.
- CNF Formula: $(x1 \vee \neg x2 \vee \neg x3 \vee \neg x4) \wedge (\ldots \vee \ldots) \wedge (\ldots) \wedge (\ldots \vee \ldots)$
- 3-CNF Formula: $(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\ldots \vee \ldots \vee \ldots) \wedge \ldots$
- 3-CNF-SAT is in *NP* problem and solve it by:
  - Guess truth assignment $(x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_4}) \wedge (\overline{x_1} \vee x_3 \vee x_4)$
  - Check assignment to see if it satisfies CNF formula
- Example: Truth assignments

X_1	X_2	X_3	X_4	Expression
T	T	T	T	?
T	T	T	F	?
…	…	…	…	…

… how many more?

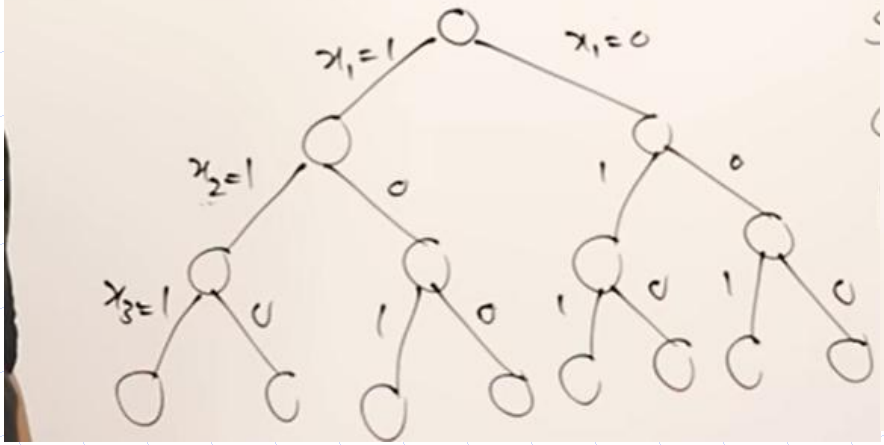
- Checking phase: $\Theta(n)$

CNF-Satisfiability — 2^n

$x_i = \{x_1, x_2, x_3\}$

$CNF = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3)$

C_1 C_2



Satisfiability — 2^n

Expontial Time

0/1 Knapsack — 2^n

Traveling SP — 2^n

Sum of Subsets — 2^n

Graph Coloring — 2^n

Hamiltonian Cycle — 2^n

Solving NP-complete problems

- If your problem is NP-complete, then do not waste time looking for an efficient algorithm.
  - Optimal solution is intractable.
  - Unlikely to find a polynomial-time algorithm for solving it exactly.
- Instead look for heuristics and approximation algorithms.

Solving NP-complete problems

- Heuristics:
  - Heuristics may produce good solutions but do not come with a guarantee on the quality of their solution.

- Approximation algorithms:
  - Algorithms provide near-optimal solutions to NP-Complete problem. Near optimal is good enough for many problems.
  - Algorithm producing a solution that is guaranteed to be within some factor of the optimum